

HISTORY ALTERATIONS IN PUBLIC GIT REPOSITORIES

4.1 Introduction

Chapter 3 established that the named references developers use to designate software releases—tags—are mutable in practice: they can move to a different commit, disappear entirely, or be recreated with different targets while the version string they carry remains familiar to downstream consumers. That chapter also noted, at its close, that mutable release labels represent only one surface of repository mutability; the other surface is the commit graph itself. A tag can remain perfectly stable while the commits it ultimately reaches through its ancestry have been silently replaced. This chapter moves the investigation inward, from the boundary of release names to the reachable commit history that public branches advertise at each archival observation.

*C'est un
résumé pas
une
transition
(faux se)*

Distributed version control systems (DVCS) such as Git are, for the most part, append-only in ordinary use: developers add new commits and exchange them by pushing to and pulling from public repositories [12]. Yet Git also permits altering previously recorded history [12], through operations such as amend, rebase, and filter-repo, interactive squashing, and force-pushed replacements of branch tips; developers routinely use these techniques to polish unshared branches before publication. The same operations on shared history create a different class of integrity questions: commit identities, parent links, authorship fields, and timestamps are routinely treated as an audit trail, yet they are not append-only facts about a repository; they are whatever the current object graph says after the latest rewrite.

motivation ?

History alterations on public branches are problematic for two converging reasons. First, they break the pull/push workflow, which relies on the stability of commit identifiers as checkpoints for history exchanges. Second, by breaking commit identities, alterations create opportunities for supply-chain attacks: alongside a legitimate history rewrite, an attacker can sneak in malicious

*présenter comme
une autre feuille
de la SC. Faire
une autre simulation
à 3.*

changes that are difficult to audit because of the confusion caused by commit-identity modifications [36, 56]. Software Heritage mitigates the observability problem that would otherwise make rewriting invisible: successive crawls retain prior object identities even when forges overwrite refs [3, 15]. That archival property makes it possible to ask, at ecosystem scale, how often public histories change in ways that remove or replace commits, how those events distribute across branches and projects, and whether the file-level footprints resemble benign hygiene (for example, cleaning credentials) or governance-sensitive patterns (for example, retroactive license edits).

4.1.1 From Mutable Releases to Altered Histories

Tag mutability and history rewriting are not competing explanations for the same failure; they are composable sources of drift. A project can keep every release tag stable while still replacing the commits those tags eventually point through, or rewrite metadata underneath a stable tree snapshot. Conversely, a moved tag is already a signal that some operation that changed refs occurred; the metadata-only tag moves measured in Chapter 3 are consistent with workflows that rewrite history and preserve directory identity while altering revision identifiers [51]. This chapter therefore completes the empirical picture begun in Chapter 3: it quantifies mutability inside the ancestry that public heads advertise, independent of whether any particular tag ever moved.

4.1.2 Why History Rewriting Deserves Broader Study

History rewriting matters for supply-chain and assurance arguments for three converging reasons.

First, **downstream workflows depend on stable graph identities**. Forks, clones or signed-release pipelines all assume that fetching a branch name yields a coherent evolution of commits unless a maintainer explicitly announces a disruptive operation. Force-pushed rewrites break that assumption silently for actors who are not continuously mirroring.

Second, **metadata is itself a channel**. Author timestamps, names, and messages feed empirical software engineering, review analytics, and security heuristics; when they can change retroactively, analyses that treat metadata as ground truth inherit a structural blind spot unless they anchor to independently versioned snapshots [19].

Third, **the same mechanism serves remediation and concealment**. Removing accidentally committed credentials or correcting license files is legitimate and common; it is also operationally similar, at the object-graph level, to hiding malicious commits unless independent archives or hash-disciplined consumers keep contradictory evidence [36].

What this chapter establishes. The chapter extends the underlying ASE 2025 study [49] along five complementary lines. Section 4.2 and Section 4.3 establish prior-work context and the archival comparison pipeline. It then quantifies how often publicly archived Git histories

- Motivation sur autre ph ≠ ditag
- les altérations
- pourquoi est-ce un problème difficile
- plan

retourner
à C3
fréquents

est-ce similaire
à C3
begin de
faire cela
ici
motivation
disruptive
avant

au contraire tu demandes l'hygiène
que ce n'est pas

change between successive observations and how those events distribute across repositories (Section 4.4.1). At the branch level, it identifies which branches carry the largest share of alteration events and how the likelihood of rewriting relates to repository visibility on GitHub (Sections 4.4.2 and 4.4.3). At the commit level, it applies a taxonomy that separates metadata-only rewrites from directory and file changes, and reports how alteration types vary between the full corpus and highly starred projects (Section 4.5). At the security and governance level, it examines recurrent file-level patterns and displays two case studies in depth: retroactive license changes and removals of files whose names suggested secrets that remain recoverable from the archive (Section 4.6). Finally, it packages the detection pipeline into GitHistorian, an operational tool that can help maintainers and auditors spot alterations in repositories of interest (Section 4.7). The chapter closes with interpretation (Section 4.8), threats to validity (Section 4.9), and a conclusion (Section 4.10).

4.2 Related Work

4.2.1 History Alterations: Motivations and Handling

Git's data model separates content-addressed objects from mutable refs; anything reachable only through a ref can disappear from the server's advertised graph after a rewrite [12]. Reference material treats history editing as a normal local maintenance tool—the Pro Git book lists modifying files, changing commit messages, and reordering, squashing, splitting, or removing commits as actions one can perform before sharing history—while warning that shared branches require coordination, precisely because rewrites change commit identities and disturb collaboration invariants [12]. Developers use alterations to maintain “clean” histories that ease code review and project management [9, 32], and tooling encourages squashing to improve clarity [27, 34, 45]; research on collaborative workflows has proposed taxonomies that weigh the pros and cons of rebasing [45, 52], and has studied the resolution of merge conflicts that arise after rebases [32]. The ability to modify files is also what makes history rewriting attractive for removing erroneously committed sensitive information—a use case studied directly in Chapter 5.

Limited tooling exists for recovering missing elements from commit history via inference rather than archival comparison. AST-based approaches attempt to detect file renames with high accuracy by focusing on content structure rather than metadata [20]. Probabilistic methods have been developed to track file movements between versions—addition, removal, and relocation—using nearest-neighbour clone detection, but without capturing metadata alterations [38]. Neither line of work operates at the scale of an independent archive, and neither can reconstruct commits that have already been removed from the advertised graph.

fin de l'intro → un peu long

les intro sont souvent complexes.

phrase intro pour expliquer la structure

étrangé d'avoir la motivation ici!

RW!

pas motivé

les trucs de RW

c'est peut-être pas au but

écrites paragraphes : ~~problem~~ ~~théorie~~ &
problem → solution → thesis conclusion

CHAPTER 4. HISTORY ALTERATIONS IN PUBLIC GIT REPOSITORIES

4.2.2 Empirical Analysis of History Alterations and Archival Comparison

There are almost no empirical studies of history alterations, in part because of the difficulty noted above: after a rewrite, the prior history no longer exists and cannot be compared with what remains. The most relevant prior work subjected the Linux kernel and its contributing repositories to continuous snapshots and found that the actual development history of DVCS projects can be irretrievably lost; that study examined alterations occurring across repositories, including rebasing and metadata changes, but did not examine alterations within individual repositories nor identify specific file-level changes at large scale [21]. Empirical mining research has long warned that Git histories are not a neutral ground truth: early MSR work catalogues structural and statistical pitfalls [9], GitHub-scale studies echo that measurement and sampling are fragile under real platform dynamics [33], and a study of timestamp anomalies in a subset of Software Heritage suggests that history alterations affect timestamp reliability and can compromise dataset integrity [19]. Security research complements those caveats with explicit threat models: metadata manipulation can omit commits, misrepresent merge outcomes, and steer developers toward unsafe actions, motivating signed logs of developer-visible repository state [56].

Large-scale studies of public development increasingly rely on compressed graph representations of version-control data to make global queries feasible [10, 47]. Software Heritage has been positioned both as a preservation effort and as a dataset for provenance and reproducibility research because it retains objects and snapshots across upstream churn [3, 15]. The altered-histories methodology exploits that property directly: consecutive snapshots of the same origin are differenced, root-cause commits are identified, and alterations are classified along metadata-versus-directory axes. The stance is forensic (compare two observed worlds) rather than inline cryptographic [56], because Git itself does not provide append-only project history to downstream clients.

4.2.3 Research Gap and Positioning of This Chapter

To our knowledge, no prior work had characterised, quantitatively or qualitatively, the extent of history alterations in public code repositories, let alone at large scale. This chapter fills that gap: it provides a taxonomy for categorising alterations, digs into two recurrent patterns (secret suppression and retroactive license changes), and provides an automated tool to help developers detect alterations in repositories of interest [49, 50]. The study establishes that public version-control state should not be assumed permanently stable once a repository is published.

My work on this topic spans two complementary empirical studies in this thesis. In Chapter 3, I measured tag alterations—deletions and moves of references that downstream users and tools often treat as stable release anchors—and connected archive-scale prevalence to package-level consequences for dependency resolution, release identification, and rebuildability, including hash-disciplined ecosystems where integrity failures become explicit build errors

tu devrais avoir 2 axes de RW

1) extraction / detection 2) Analyse taxonomique

rather than silent drift [4, 51]. The present chapter differs from that earlier study in two ways. First, whereas the tag study isolates tag alterations specifically, this chapter widens the lens to reachable commit histories: rather than asking whether a tag still resolves to the same commit, it asks whether commits once advertised on public branches disappear or are replaced across successive archival observations. Second, this chapter foregrounds consequences for forensics and repository evolution—commit identities, parent links, authorship metadata, and file-level footprints—whereas the tag study foregrounds reproducibility and supply-chain integrity; rewritten tags matter directly for dependency resolution, release identification, and rebuildability, whereas rewritten branches and commits matter also for audit trails and repository evolution. Together, the two studies isolate two reference types through which mutability enters downstream workflows—release names and ancestry—that actors routinely conflate when they record only a tag or branch name. GitHistorian, discussed in Section 4.7, packages the snapshot-differencing idea developed here for maintainers and auditors who need actionable reports rather than global statistics alone [50].

4.3 Methodology

This study shares the observational stance established in full in Chapter 3. The four-phase structure is also shared on some level: collect a large corpus of periodically crawled repositories, detect alterations by snapshot comparison, categorise them with a taxonomy, and analyse recurrent file-level patterns through targeted case studies. The rest of this section describes only what differs from the tag study: the unit of differencing is the reachable commit set rather than resolved tag targets, the corpus and filters differ in their specifics, and the taxonomy and case studies are built for commit-level rewrites.

4.3.1 What Differs from the Tag Study

The tag study compares the resolved targets of tag references; this study compares the set of commits reachable from public branch heads. The Git data model is a Merkle directed acyclic graph (DAG) in which each node (commit or other artifact) is identified by a cryptographic checksum computed recursively over its content and metadata, including the identifiers of parent commits. Because of this construction, modifying any part of a commit changes its identifier, and—through a snowball effect—changes the identifiers of every commit that transitively references it. Two common rewrite operations illustrate this: amend retroactively modifies a recorded commit (its metadata and/or content), and rebase merges, splits, or reattaches commits, both are illustrated in figure 4.1.

In both cases the old commits disappear from the advertised graph and new ones replace them, while Software Heritage still reaches the old commits from the previous snapshot of the same origin, as shown in figure 4.2.

en fait, je ne ⁵¹comprends pas pourquoi tu devais alors que toute la recensement apporte qq chose de très différent

non qu'on
c'est bcp +
ou
+

uladevaloux le
chapitre comme un
add-on

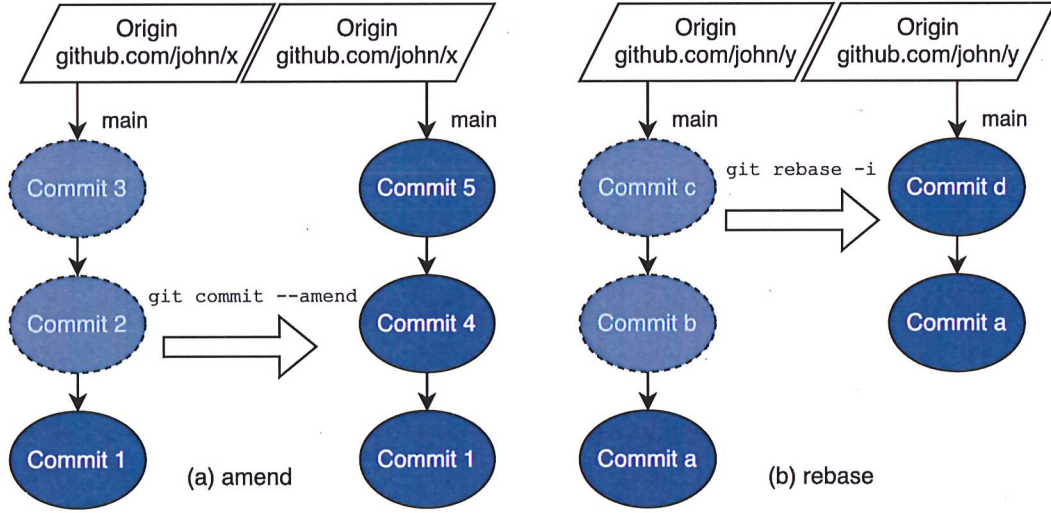


Figure 4.1: Common scenarios leading to history alterations in Git: (a) commit amendment, (b) interactive “rebase”. Grayed out commits are no longer reachable after alteration. Arrows point from newer to older commits.

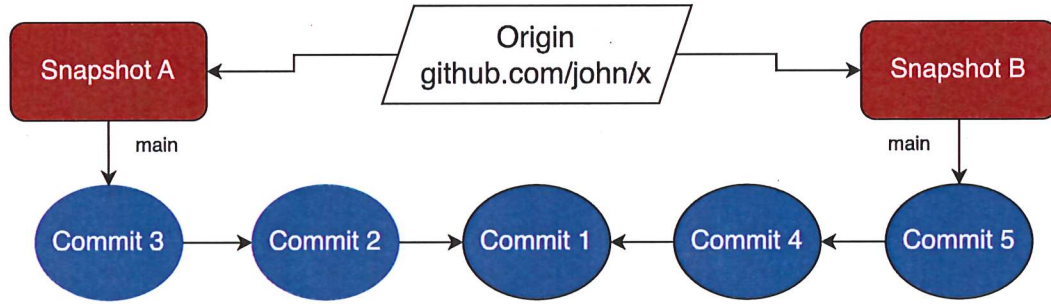


Figure 4.2: Repository state before and after `git commit -amend` of Figure 4.1(a). Both histories are preserved by Software Heritage and share common commits.

This yields the operational definition used throughout: a repository R underwent a *history alteration* if there exist two subsequent snapshots S_1, S_2 of R such that at least one commit present in S_1 is missing from S_2 . An *altered commit* is one present in an earlier snapshot but missing from a later one. Because of the snowball effect, a single alteration can affect many commits, so for each alteration the analysis identifies one or more *root-cause commits*: the oldest commits, in topological order, among all altered commits—those are unequivocally directly altered rather than changed as a consequence of an earlier modification.

4.3.2 Data Collection and Repository Selection

The corpus starts from the Software Heritage graph dataset, timestamp 2024-08-23, which contains the VCS history of more than 310 M repositories and packages (“origins”) [47]. To avoid

under-estimating the ratio of repositories that witness alterations, origins visited fewer than two times are excluded (199 M), as are non-Git origins (19 000 thousand), leaving 111 M origins where evidence of history alterations can potentially be found. As in the tag study, popularity-dependent analyses that relate alteration likelihood to repository visibility use GitHub star counts made available by Software Heritage and are restricted to GitHub repositories, while all other analyses use every archived origin regardless of platform; star counts can be inflated but still indicate visibility and scrutiny-worthiness [31].

4.3.3 Detection Framework and Branch Unification

For each origin with at least two visits, and for each Git branch in them, all pairs of successive snapshots are compared to identify commits from the earlier snapshot missing from the later one (figure 4.3 step 1); within that set, root-cause commits are extracted per the definition above (figure 4.3 step 2). This step is implemented with custom code that mines the compressed graph representation of the Software Heritage archive locally, via its Rust API [10]. The framework counts alterations at repository scale and records branch-level patterns for the distribution analyses in Section 4.4.

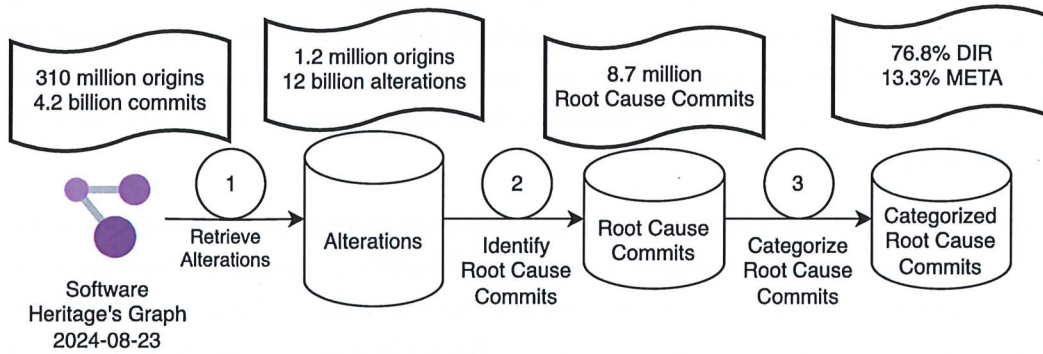


Figure 4.3: Methodology for detecting and categorizing history alterations.

Because different branch naming schemes denote the same concept, branches are unified before aggregation (Table 4.1): main and master become main; dev/devel/develop/development become development; pull/<number>/head becomes pull request; and renovate/<anything> becomes renovate.

Table 4.1: Branch-name unification used before aggregating alteration statistics.

Branch names or patterns	Unified name
main, master	main
dev, devel, develop, development	development
pull/<number>/head	pull request
renovate/<anything>	renovate

4.3.4 Taxonomy of Altered Commits

Root-cause commits are categorised with a custom taxonomy (Table 4.2 and step 3 of figure 4.3); no preexisting taxonomy for this existed in the literature, except for the one we used in Chapter 3 that comes close to it. It was developed via an iterative mixed method: sampling uncategorised alterations, manually comparing before/after states, capturing the individual data that differed (introducing new categories as needed), applying all categories to the whole dataset, and repeating until everything was categorised. The categories are:

- **Different Branch Name:** content and metadata are unchanged, but the commit is now found only in a different branch than before—the simplest case.
- **META:** the commit was replaced by a new commit with strictly identical file and directory content but different metadata; this is refined to indicate which metadata fields changed (author, message, date, committer, committer date, other), and multiple fields can change at once.
- **DIR:** the commit's file or directory content was altered or deleted, in part or whole; metadata changes are *not* analysed here because there is no single reference commit to compare against. DIR is refined into *Content Split* (each file that existed in the altered commit is still present but split across several other commits; exclusive of the other DIR subcategories), *File Modified* (at least one file modified), and *File Removed* (at least one file removed and not found elsewhere).

To distinguish these, the analysis looks in the newest snapshot at commits starting from the children of the parents of the root-cause commit and proceeds for ten generations (commit-tree depth). If a file path cannot be found after ten generations of descendants—or if there are no more descendants—the file is considered removed. The horizon of ten generations is a heuristic trade-off between analysis time and the diminishing likelihood that a file reappearing far from a change is genuinely the same file.

4.3.5 Case-Study Design

Two recurring patterns in the DIR category are examined in depth in Section 4.6. The first, *secret suppression*, focuses on files removed (category: File Removed) that should never have been shared via a public repository, detected by popular filenames used to store private keys or passwords, such as `id_rsa` (without the `.pub` extension) or `credentials.txt`; the full list ships with the replication package [50]. The second, *retroactive license changes*, identifies modified license files (category: File Modified) on the main branch (after unification) whose names contain `LICENSE` or `LICENCE` (case-insensitive). ScanCode, a state-of-the-art license-detection tool [2, 44], is run on the files before and after alteration, keeping only the highest-confidence result above 90 %. The resulting before/after license sets are categorised to gauge the magnitude of change:

4.4. PREVALENCE AND DISTRIBUTION OF HISTORY ALTERATIONS

Table 4.2: Root-cause commits by category. Metadata fields within META are not mutually exclusive.

Category	Field	#Commits
META (1 160 725)	Author	667 424
	Message	691 060
	Date	769 287
	Committer	649 937
	Committer Date	864 607
	Other	55 638
DIR (6 693 247)	File Modified	5 469 080
	File Removed	1 287 304
	Content Split	763 369
Different Branch Name	—	866 113

License Update (the set of unversioned licenses is unchanged but a versioned license changes, e.g. GPL-2 → GPL-3), *Partial Change* (some unversioned licenses changed, e.g. GPL added, MIT removed), and *Full Change* (all unversioned licenses changed).

4.4 Prevalence and Distribution of History Alterations

4.4.1 Frequency Across Repositories

Across the entire corpus, the study observes 12.5 B alterations spread across all analysed branches, corresponding to 8 720 085 (8.7 M) identified root-cause commits. In terms of repository count, 1 218 547 (1.22 M) repositories contain at least one altered commit, approximately 1.10% of all examined repositories. The event count is orders of magnitude larger than the repository count because a single maintenance campaign can touch many commits and edges, so readers should not equate “events” with “projects”. The proportion is modest in relative terms but not negligible in absolute ones: users of more than one million repositories can—and, assuming future stability, will—use public Git repositories that underwent history alterations during their lifetime. At the same time, the low percentage indicates that history rewriting remains an exception rather than standard practice, suggesting that common Git usage adheres to the expectation of immutable histories.

4.4.2 Distribution Across Branches

Alterations are not confined to obscure sandbox branches. After the unification of Table 4.1, the top branch categories by share of alteration events are listed in figure 4.4: pull-request branches dominate (37.57 %), reflecting interactive rebasing and commit squashing to clean up history before integration.

However, `main` (after unifying `main/master`) alone accounts for 11.39 % of events—ranking second and rejecting the myth that rewriting is only a pull-request hygiene issue.

The third category, `renovate` (9.56 %), reflects automated dependency-update bots that rewrite commits as proposals evolve before merging, while development branches account for 0.88 %. Even though roughly half of all alterations follow a known practice, alterations on `main` branches compromise fundamental version-control assumptions, potentially breaking downstream dependencies, invalidating signed commits, and disrupting reproducible builds [13].

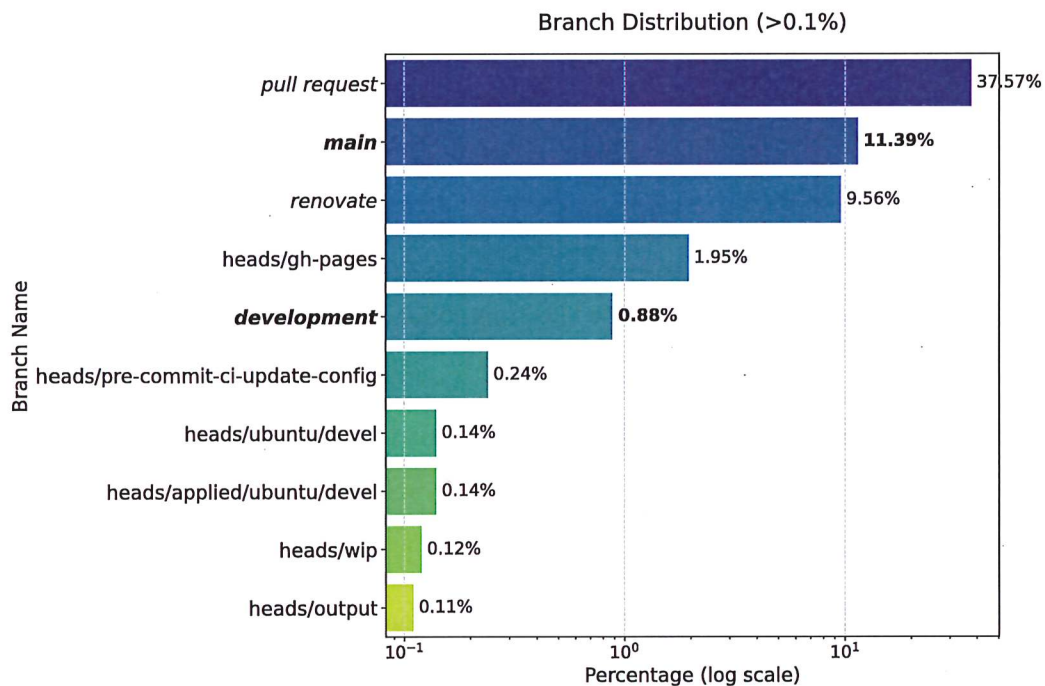


Figure 4.4: Most impacted branches by Altered Histories (> 0.1%)

4.4.3 Distribution Across Repositories

Joining the 1.22 M altered repositories with their GitHub star counts (after removing 283 203 origins with unknown star counts) gives the following distribution shown in figure 4.5: 0–1 star, 676 252 origins (72.3%); 2–9 stars, 115 431 (12.3%); 10–99 stars, 82 519 (8.8%); 100–999 stars, 43 416 (4.6%); and 1000+ stars, 17 726 (1.9%).

The proportion of origins with a detected alteration in each bucket reveals a pronounced relationship between popularity and alteration likelihood: 0.3 % for 0–1 star, rising to 1.6 % for 2–9, 3.0 % for 10–99, and stabilising around 3.4 % for 100–999 and 3.3 % for 1000+. It is therefore *not* the case that less popular repositories experience more rewrites; rather, as repositories gain visibility, the likelihood of history modification increases substantially, and even mature, popular repositories (100+ stars) experience a non-negligible amount of alteration. Within very

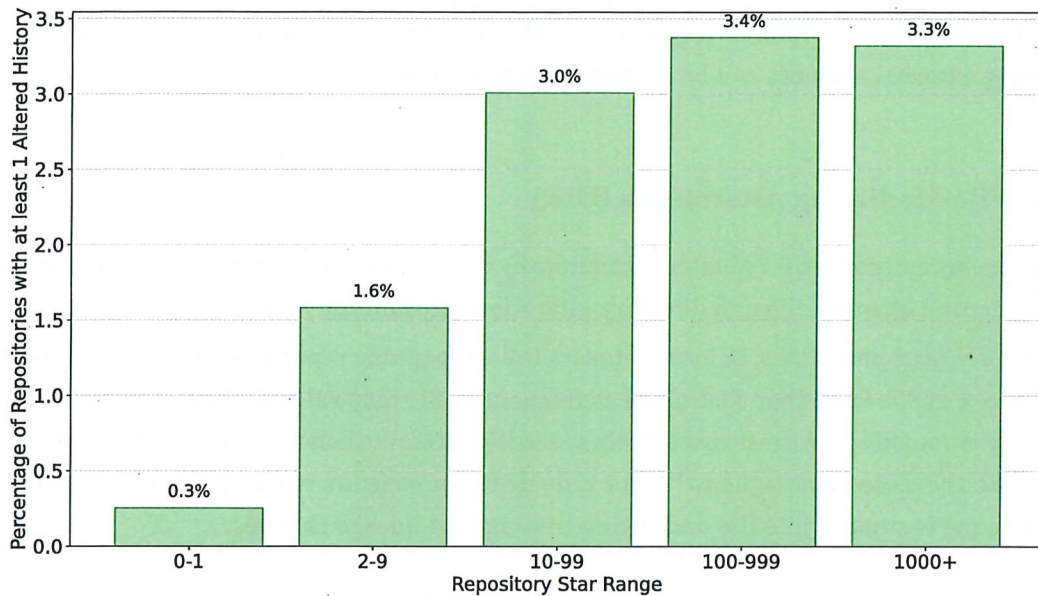


Figure 4.5: Proportion of Repositories with Altered Histories per Star Range

popular repositories (100+ stars), alterations happen mostly on pull-request branches (56 %) but remain non-negligible on main branches (1.8 %), with implications for dependency management, reproducible builds, and the reliability assumptions developers make when incorporating external repositories.

4.5 Taxonomy of History Alterations

aptes extraction → analyse

Applying the taxonomy of Table 4.2 to the 8.7 M identified root-cause commits gives the category distribution summarised in figure 4.6, reported for the whole corpus and for the subset of repositories with 1000+ GitHub stars.

4.5.1 Metadata-Only Alterations (META)

Among the root-cause commits, 1.2 M instances (13.3%) exhibit modifications only to commit metadata, leaving files and directories unaltered. The committer date is the most frequently modified attribute, across both the complete dataset and the 1000+ star subset, and metadata-modification patterns are uniform regardless of popularity.

A co-occurrence analysis shows that when two fields change at once (figure 4.7(a)), the most frequent pair is the author date together with the committer date—changing one without the other would be surprising. The most frequent triplet (figure 4.7(b)), occurring 85 % of the time in the global dataset, is a commit-message change accompanied by changes to the author and committer dates, the expected behaviour when editing a commit message. Such events still

rapprocher

break naive audit assumptions: any workflow that logs “who landed commit abc1234” without anchoring external evidence can be retroactively contradicted.

4.5.2 File-Modifying Alterations (DIR)

The taxonomy categorises 6.7 M altered commits by changes to file or directory content; combined with structural changes, file and directory alterations account for 76.8% of root-cause commits.

File Modified dominates in both datasets (all vs. popular repositories), exceeding 60 % of occurrences, and is more than four times as frequent as file removal: a developer who alters a file in history is roughly four times more likely to modify it than to delete, rename, or move it. These events are the closest analogue to “silent code drift” for consumers who track branch tips: the branch name is stable while the underlying tree-content lineage changes.

4.5.3 Structural and Reference-Level Changes

The *Different Branch Name* subcategory accounts for 866 k commits whose content and metadata are unchanged but that are now found only in a different branch. The most plausible explanation is the merge of so-called feature branches that are later removed; this is still problematic for direct users of those branches, though understandable when the branches are explicitly documented as volatile. Grouping these phenomena alongside file and metadata signals ensures that prevalence tables do not conflate “tree stable” with “history stable”.

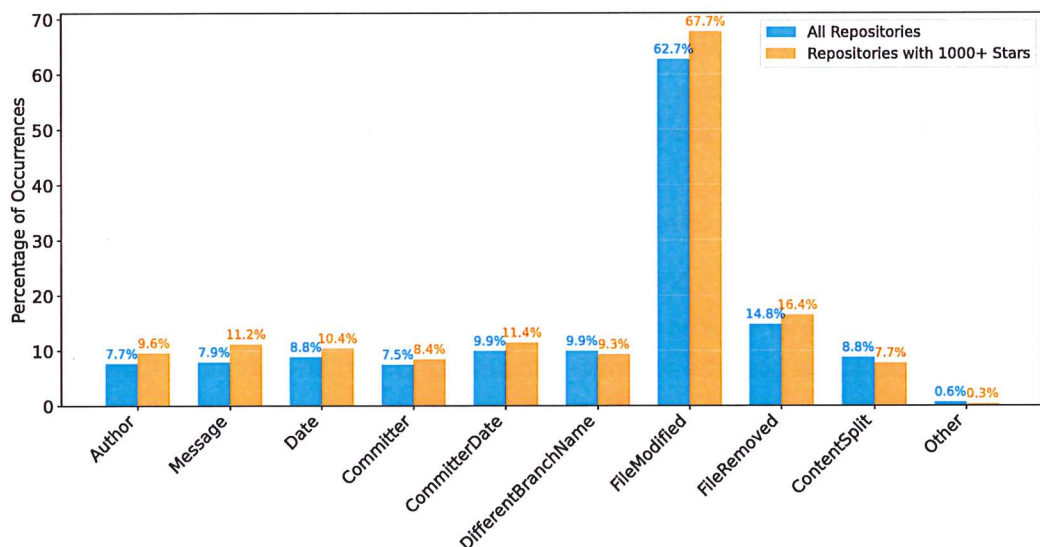


Figure 4.6: Distribution of Categories: All vs. Popular Repositories

4.6. SECURITY-RELEVANT CHARACTERISTICS OF ALTERED HISTORIES

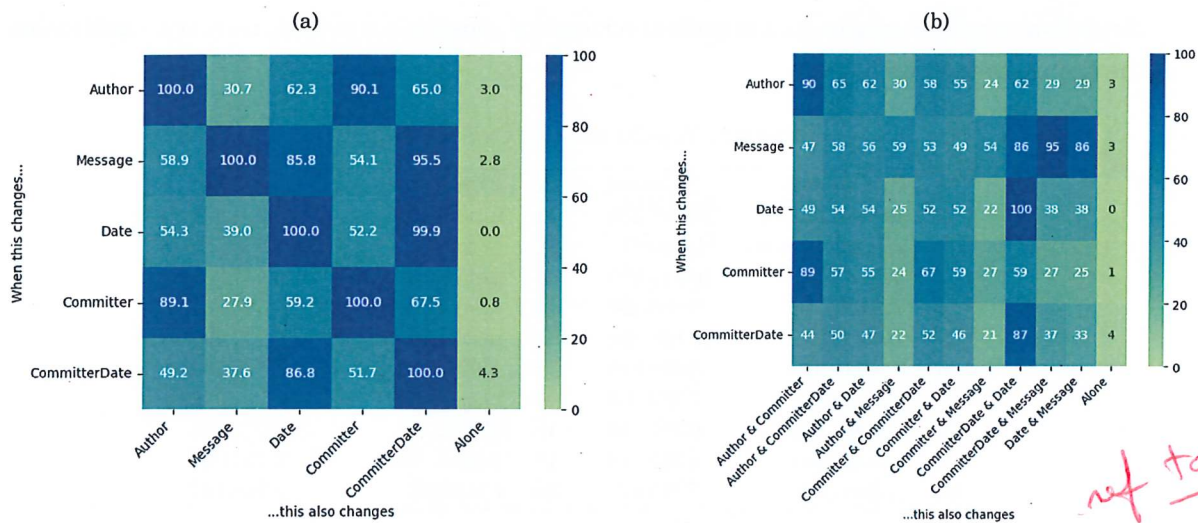


Figure 4.7: Co-occurrence analysis of metadata alteration (a) Pair (b) Triplet

TODO Add mask for pair in triplet and 100% in pairs

4.5.4 Other Observed Alteration Patterns

Automation-heavy namespaces—for example, dependency-update bots such as Renovate and Dependabot—contribute recurring rewrite shapes that are operationally benign but still appear as alterations in the archive. The taxonomy therefore supports separating human-maintained default branches from bot-owned integration lines when interpreting the branch tables in Section 4.4.

4.6 Security-Relevant Characteristics of Altered Histories

4.6.1 Recurrent File Patterns

Because file alterations are the most common form of rewriting, an obvious question is which files are modified or removed. Extracting the filename part of all paths involved in DIR alterations yields the top-20 most commonly altered filenames in Table 4.3. The most frequently altered files are predominantly configuration and build-related artifacts: `default.nix`, `package.json`, `Makefile`, `pom.xml`, `CMakeLists.txt`, and `APKBUILD` represent different build systems and package managers, suggesting that dependency updates, version bumps, and build-configuration changes are primary motivations for rewriting. The high frequency of `default.nix`, `package.json`, and `meta.yaml` matches the workflows of automated dependency tools (Renovate, Dependabot, Nix update bots) that modify these files and then rewrite commit histories. Documentation files (`README.md`, `CHANGELOG.md`, `index.md`) and ecosystem entry points (`index.js`, `index.html`, `index.ts`, `__init__.py`) also appear frequently, while `LICENSE` (#20) and `metadata.xml` (#13) in-

dicating that legal-compliance and metadata corrections constitute a notable category—motivating the license case study below.

Table 4.3: Top-20 most altered file names.

#	File name	Count	#	File name	Count
1	default.nix	13401935	11	CHANGELOG.md	5001971
2	package.json	12594717	12	__init__.py	4859517
3	Makefile	10615195	13	metadata.xml	4477947
4	README.md	9969599	14	CMakeLists.txt	4380963
5	index.js	9129894	15	package.py	4336900
6	Manifest	8891876	16	APKBUILD	3375064
7	pom.xml	7080355	17	template	3122049
8	index.html	6297643	18	BUILD	3018131
9	meta.yaml	5425074	19	index.ts	2764520
10	index.md	5372929	20	LICENSE	2762147

4.6.2 Case Study: Retroactive License Changes

The first case study focuses on retroactive license modifications, where developers alter history to change the content of license-denoting files. The study identifies 796972 altered license files across the main branches of the dataset, spanning 32169 repositories, 76 of which have 1000+ GitHub stars. Of these, 719196 instances involved renaming, relocation, or deletion, while 77776 represented content modifications.

Using ScanCode, the before/after license sets were identified for 65688 alterations: 79% are relatively minor license updates (at most a change in license version, with copyright/author edits common), 14% represent full license changes (including transitions from permissive licenses such as MIT to restrictive ones such as GPL and vice versa), and 5% are partial changes, the last attributable to the limited number of repositories (8024 alterations) with multiple license families.

These findings are concerning for downstream users: while most open-source licenses are legally irrevocable, benefiting from the granted rights requires practical access to a version of the software released under that license. Retroactively changing version history can make old versions, under an old license, disappear from public circulation—inhibiting previously available rights, an especially acute concern for industrial users who expect legal stability.

4.6.3 Case Study: Removing Secrets from History

Across all repositories, the study identifies 13 M history alterations categorised as File Removed that involve files whose names commonly denote private information (private keys, certificates, passwords), spanning 75 k distinct repositories. Generic naming patterns are the most frequent: files containing “key” and “secret” account for 6.9 M and 805 k altered files respectively. Manual examination of a representative random sample confirmed that removed files contained sensitive

4.7. GITHISTORIAN: AN OPERATIONALIZATION OF HISTORY ALTERATION DETECTION

information, in files such as `secret.yaml` (authentication credentials) and `samlKey.jks` (keys for deploying production environments). SSH private keys are less common—probably because platforms such as GitHub can block pushes containing them—but are not absent: 108 removed files contained RSA private keys, spanning 23 origins.

These findings show that despite attempts to remediate exposure through history alteration, sensitive information remains recoverable through archival history digging: removing a secret from history is only partial remediation, and complete restoration requires regenerating or rotating all exposed credentials—at which point there is arguably little point in rewriting history afterwards. Chapter 5 takes up this thread directly, starting a study on removed files containing secrets and whether or not those secrets remain active.

4.6.4 Interpretation of Security Relevance

The prevalence of removals of files whose names suggested secrets is consistent with post-leak history rewrites—the scenario the last case study targets—though filename heuristics alone cannot exclude removals driven by refactoring, false-positive path matches, or rewrites whose primary purpose was unrelated to credential exposure. That operation targets the canonical repository view, not the full blast radius of exposure—clones, forks, and archival snapshots may already retain the removed objects, as the case study above shows—and removal counts alone cannot establish whether credentials were rotated, how long they remained public, or whether downstream copies were accounted for. License changes raise complementary governance questions: recipients may rely on SPDX headers or LICENSE files for compliance, but those files can change retroactively in rewritten histories even when branch names stay constant.

4.7 GitHistorian: An Operationalization of History Alteration Detection

Archival counts establish prevalence; practitioners still need *local* explanations—what disappeared, what replaced it, and which branch heads witnessed the change. History alterations otherwise go unnoticed except by downstream users who happen to have retrieved a repository version before the alteration and then pulled after it with branch tracking in place—conditions few users satisfy, since CI/CD pipelines often perform a fresh clone, making detection impossible. GitHistorian is a tool that detects, describes, and (if desired) helps avoid repositories that witnessed history alteration.

4.7.1 Design

The design satisfies three core requirements: scalability for analysing thousands of repositories, accuracy in detecting various alteration types, and usability for both interactive analysis and

Donner l'impression
que l'expéditeur
a changé qui
à 25

ne comprend pas
le point!
rapport avec
6.6.3

motiver!
dire
pourquoi

donc que soit mainte
nuer l'historique

automated monitoring. The architecture has four components: (1) a PostgreSQL database storing preprocessed history-alteration data derived from Software Heritage snapshots; (2) a command-line interface for interactive analysis; (3) a caching mechanism to optimise repeated queries; and (4) Git-hook integration for continuous monitoring. The local database is required because Software Heritage does not currently expose a remote API answering the queries needed to detect history alterations; this requirement could be lifted in the future if the archive operators or a third party provided such a service over the periodically published datasets. The detection mechanism queries the database for known alterations and supports branch-specific analysis (main branches only, development branches only, or all branches), and for automated monitoring it installs Git hooks that trigger checks after repository updates (post-merge) and branch switches (post-checkout).

4.7.2 Implementation and CLI

GitHistorian is implemented in Rust, using the `sqlx` crate for type-safe database interactions and `tokio` for asynchronous operations, enabling efficient handling of datasets containing billions of alteration records. The main functionality is exposed through CLI commands: the `load` command ingests the dataset into the local database with parallel processing; the `check` command audits a specific repository, querying the local database and formatting results for human consumption (with optional restriction to main or development branches); the `attach` command installs Git hooks that call `check-cached`, a cached variant that keys results by repository URL, target branch, and dataset version to avoid redundant queries. Cache validity is determined by dataset-version comparison, so results remain accurate as new alteration data becomes available while repeated queries return immediately.

4.7.3 Example Outputs and Usage Scenarios

A maintainer can check whether a repository’s history was altered, then attach the tool for ongoing monitoring, as illustrated in Figure 4.8. Documented scenarios include pre-merge auditing of third-party forks, periodic organisation-wide scans, and CI gates that alert when unexpected rewriting appears. The verbose output reports each altered commit’s branch, SWHID, snapshot destination, and subcategory, and surfaces modified file paths—information that can be crucial for security auditing (for example, when a security-related file such as `src/security/auth.py` was modified in an altered commit).

4.7.4 Practical Usefulness and Limitations

GitHistorian shifts the cost of *spot checks* from forensic expertise to tooling, but it inherits archival limits: if Software Heritage never captured the pre-rewrite state, there is nothing to diff against.

```

$ git-historian check https://github.com/example/project \
  --branch main --verbose
Connected to the database!
Found 2 altered history records for
  'https://github.com/example/project'
Record #1:
  Branch Name: refs/heads/master
  Altered Commit: swh:1:rev:a1b2c3d4e5f6789...
  Snapshot Destination: swh:1:snp:abcd1234...
  Sub Category: FileModified
Record #2:
  Branch Name: refs/heads/dev
  Altered Commit: swh:1:rev:f6e5d4c3b2a1098...
  Sub Category: CommitterDate

$ git-historian attach . --branch main --verbose
git-historian successfully attached to repository
Hooks installed:
- post-merge (triggered after git pull)
- post-checkout (triggered after git checkout)

```

FIGURE 4.8. Representative GitHistorian session: auditing a repository for alterations and attaching Git hooks for continuous monitoring. SWHIDs are illustrative.

It also does not, by itself, classify intent; it surfaces *that* rewriting occurred and *what* classes of tree or metadata edits accompany it.

4.8 Discussion

pourquoi y a-t-il discussion : objectif de cette section !

4.8.1 Implications for Software Development Practices

The findings challenge conventional wisdom about version-control best practices. The paradoxical correlation between popularity and alteration frequency suggests that current workflows may be fundamentally at odds with the immutability principle underlying version control. While alterations on pull-request and dependency-management branches follow accepted practice, their normalisation may create cultural acceptance that inadvertently extends to main branches. The substantial presence of main-branch alterations is a systemic challenge: beyond immediate impacts on CI/CD systems and semantic versioning, these practices undermine the cryptographic integrity guarantees that many security and compliance frameworks depend upon, so organisations relying on commit signatures or audit trails may unknowingly operate with compromised assurance levels.

il ne faut pas confondre discussion et bavardage
il faut un objectif à la discussion

CHAPTER 4. HISTORY ALTERATIONS IN PUBLIC GIT REPOSITORIES

4.8.2 Security and Compliance Implications

les versions précédentes

The secret-removal analysis reveals organisational security-governance issues beyond immediate credential exposure: the prevalence of post-hoc secret removal suggests systematic failures in preventive measures, pre-commit hooks, developer training, and secure workflows. More concerning is the implicit assumption that history cleaning provides adequate remediation; the detection capabilities here demonstrate that “deleted” secrets remain discoverable through historical analysis, yet many organisations may believe their incident response is complete after a rewrite. The licensing case study exposes novel legal risks: retroactive license changes create ambiguous precedents where different project versions operate under potentially conflicting terms, introducing intellectual-property uncertainty that traditional license scanning cannot detect.

4.8.3 Tool, Ecosystem, and Research Implications

The automation-driven nature of many alterations reveals a disconnect between tool capabilities and user understanding: modern dependency-management systems normalise history rewriting as an implementation detail, which may encourage its casual application in inappropriate contexts. Security tooling faces the challenge of distinguishing legitimate maintenance from suspicious activity, requiring context analysis beyond simple pattern matching. The research ecosystem faces a methodological challenge as well: significant portions of repository-based studies may analyse sanitised rather than authentic development data, with implications for the reproducibility of software-engineering research that relies on commit-based metrics and temporal analysis.

que retenir? proportion?

4.8.4 Broader Research and Community Implications

The prevalence of history modifications raises a methodological concern for the software-engineering research community that extends beyond the security applications discussed above. Repository-based studies—mining commit timelines, measuring development velocity, training defect prediction models, or constructing contributor networks—implicitly treat the commit graph as a faithful record of development history. The measurements reported here suggest that a non-trivial portion of that data may reflect sanitised or restructured versions of the original development sequence rather than authentic event logs; researchers who mine repositories without anchoring to independent archival snapshots inherit this uncertainty. This blind spot is especially consequential for longitudinal studies and for research that relies on commit-based metrics and temporal analysis, where the order, authorship, and content of commits are all treated as ground truth. For the broader open-source ecosystem, the findings raise questions about collaborative trust and the long-term integrity of development platforms. Developing community standards, audit

il faut
à la fois
dans un
objectif
de cette
discussion

practices, and tooling to surface and communicate the presence of history alterations represents an important area for future investment.

4.8.5 Supply-Chain Attack Obfuscation

IB plus resteur dans la chain

History alterations present a concerning attack vector: an adversary could inject malicious code while using history rewriting to erase evidence, making detection through conventional auditing extremely difficult. The documented normalisation of rewriting creates cover for malicious actors to disguise backdoors as legitimate maintenance, and the scale observed—affecting millions of repositories—suggests a significant blind spot, since traditional security measures focus on current repository states and may miss attacks that exploit the temporal dimension of version-control history. Supply-chain guidance increasingly stresses pinning and provenance; the measurements here show *why* pinning to a branch tip without a cryptographic object id is fragile, and why independent archives matter when disputes arise about what was public when. *ok*

4.9 Threats to Validity

4.9.1 Construct Validity

bizarre de le faire après la discussion !!!

History alteration is, by design, a destructive operation; observing it faithfully without information loss would require a user study on developer machines, dramatically reducing scale. This work takes the opposite approach—a very large-scale experiment relying on Software Heritage to collect and preserve snapshots that are then compared pairwise—and therefore depends on archival frequency: only alterations visible between successive visits are observable. The prevalence figures reported in this chapter should be read as a conservative lower bound on the phenomenon: missed or merged events imply undercounting, not inflation, so the true scale of history rewriting and its downstream impact on reproducibility, auditing, and supply-chain assurance may be significantly larger than the archival evidence alone establishes. Short-lived malicious or accidental alterations reverted between visits are missed, and multiple alterations between two visits of the same repository cannot be distinguished and are merged into one. Metadata changes are not detectable when file changes co-occur, due to the lack of a single reference commit to compare against; metadata could nonetheless be characterised for more than one million commits, with no claim made about the rest. The ten-commit search horizon for locating moved content is a deliberate trade-off: content might be found with a larger horizon, but a file reappearing many commits away is debatably the same file. *question*
IB
reponse

4.9.2 External Validity

Only Git repositories are considered, and no further generality to other DVCS is claimed. Software Heritage archives other DVCS as well, so the same dataset could in principle support comparative

analysis, but disentangling the specificities of each system (for example, branch naming) would be difficult, and some DVCS support non-destructive history updates that retain previous states. Finally, Git is also used as a storage backend for independent DVCS such as Jujutsu (jj), where history alterations are used more liberally; these cases were not separated from “regular” Git repositories and may affect the analysis, though they are expected to remain quantitatively marginal to date.

4.10 Conclusion

This chapter showed that history rewriting on public Git repositories is a large-scale, archivally observable phenomenon: across 111 M repositories, the study found 12.5 B alteration events and 1.22 M affected repositories (1.10%), with branch distributions that implicate default lines rather than only pull-request sandboxes. Applying a novel taxonomy, most rewriting touches files or directories (76.8% of root-cause commits), while a smaller but material share is metadata-only (13.3%), which matters for any audit process that trusts commit hashes without examining trees. Two targeted case studies showed that alterations are used to remove private information committed by mistake (for example, private keys) and to retroactively change open-source licenses; GitHistorian demonstrates that the same evidence base can feed operational tooling, not only global statistics.

Future work includes discerning legitimate from malicious alterations and designing automated detection of suspicious patterns, and studying the *timing* of rewrites—distinguishing immediate corrections from long-term exposures, and, in the secret-suppression scenario, early-stage removals (when a project was young and security practices immature) from more worrisome late-stage removals.

The most urgent open question left by the secret-suppression case study is one that this chapter cannot answer from archival evidence alone. The 13 M removals of files whose names suggested secrets detected across 75 k repositories show that developers *did* push private keys, credential files, and authentication tokens to public repositories, and then rewrote history in an attempt to erase them. But erasing a secret from the repository does not retract it from independent archives, forks, caches, or mirrors—and, more critically, it does nothing to the credential itself unless the credential is separately rotated. The secrets identified in this chapter remain recoverable from Software Heritage regardless of the rewrite; the question is whether they also remain *usable*. Chapter 5 takes up that question directly: it recovers secrets embedded in alteration-affected commits, characterises their types and exposure windows, and establishes a responsible-disclosure protocol to test whether the underlying credentials are still active after the remediation attempt.

